TERRAFORM

Interview Preparation Guide

3–5 Years DevOps Engineer Level

■ State Management	**5 Questions**
■ Drift & Apply Failures	**3 Questions**
■ Environment Design	**4 Questions**
■ Modules & Reusability	**4 Questions**
■ Import & Migration	**3 Questions**

SECTION 1 — State Management

Q1. How do you prevent Terraform state conflicts when multiple engineers work on the same project?

Use remote backends with state locking. The standard approach:

- Store state in S3 (or GCS/Azure Blob) — never local tfstate in a shared codebase
- Enable DynamoDB-based state locking so only one engineer can run apply at a time
- Use workspaces or separate state files per environment (Dev/QA/Prod)
- Enforce CI/CD pipelines as the only entity running terraform apply — not individuals

```
terraform {
  backend "s3" {
    bucket         = "tf-state-prod"
    key            = "infra/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "tf-state-lock"
    encrypt        = true
  }
}
```

■ *Without locking, two concurrent applies can corrupt state — this is one of the most common real-world issues.*

Q2. What would you do if the terraform.tfstate file gets corrupted?

Corruption is rare with remote backends but can happen. Recovery steps in priority order:

- 1. Check S3 versioning — restore the last valid tfstate version from S3 object history
- 2. If no versioning, use terraform show on any plan file or backup (.tfstate.backup) if present
- 3. As a last resort: manually reconstruct state using terraform import for each resource
- 4. After recovery, run terraform plan before any apply to confirm state matches reality

■ *Always enable S3 versioning on your state bucket. It costs almost nothing and is your best recovery path.*

Q3. How do you migrate a local state file to a remote backend?

Simple two-step process:

- Step 1: Add the backend block to your Terraform config (S3, GCS, etc.)
- Step 2: Run terraform init — Terraform detects the new backend and asks if you want to copy existing state

```
# After adding backend config:
terraform init
# Terraform will prompt:
# "Do you want to copy existing state to the new backend? (yes/no)"
# Type: yes
```

Verify with: terraform state list — should show same resources from remote backend.

■ *Do not delete the local tfstate until you've confirmed the remote state is intact and plan shows no changes.*

Q4. How do you recover from accidental state deletion?

Options depend on your setup:

- If using S3 with versioning: restore previous version of the tfstate object immediately
- If no backup exists: run terraform import for each resource manually — tedious but possible
- If using Terraform Cloud: check version history in the UI
- Post-recovery: run terraform plan to check for drift before any apply

■ *Deleted state does NOT delete real infrastructure. Your AWS resources still exist — only Terraform's knowledge of them is gone.*

Q5. How do you move resources between state files?

Use terraform state mv with the -state and -state-out flags, or use the moved block (Terraform 1.1+):

```
# CLI approach:
terraform state mv \
  -state=old/terraform.tfstate \
  -state-out=new/terraform.tfstate \
  aws_instance.web aws_instance.web

# Or use moved block (preferred, version-controlled):
moved {
  from = module.old.aws_s3_bucket.logs
  to   = module.new.aws_s3_bucket.logs
}
```

The moved block is better for team settings — it's code-reviewed, documented, and reversible.

SECTION 2 — Drift Detection & Apply Failures

Q6. terraform apply failed after creating some resources. What are your next steps?

Do NOT blindly re-run apply. Investigate first:

- 1. Read the error — identify which resource failed and why (IAM permission, quota, dependency)
- 2. Run terraform plan to see current desired vs actual state
- 3. Fix the root cause (wrong AMI ID, missing IAM policy, incorrect VPC subnet)
- 4. If a partially created resource is causing issues, use terraform taint or terraform state rm to force recreation
- 5. Re-run terraform apply — Terraform is idempotent; already-created resources won't be recreated

■ *Partial applies leave state partially updated. Always check terraform state list before and after to confirm consistency.*

Q7. A resource was deleted manually from AWS. How do you fix Terraform state?

Two valid approaches depending on what you want:

- Option A — Recreate the resource: Just run terraform apply. Terraform sees it's missing and recreates it.
- Option B — Remove from state (if you intentionally deleted it): run terraform state rm

```
# Check what Terraform sees:
terraform plan   # Will show resource as missing / to be created

# If you want Terraform to stop managing it:
terraform state rm aws_s3_bucket.logs

# If you want it recreated:
terraform apply  # Terraform will create it fresh
```

■ *Never manually modify tfstate JSON. Use CLI commands. Manual edits corrupt formatting and break state integrity.*

Q8. How do you detect and resolve infrastructure drift?

Detection:

- terraform plan is the primary drift detector — run it regularly or on a schedule in CI
- Use terraform refresh (or plan -refresh-only) to sync state with actual cloud reality
- Tools like Atlantis, Driftctl, or Terraform Cloud can automate scheduled drift detection

Resolution:

- If drift is unwanted (someone changed AWS manually): terraform apply restores desired state
- If drift is intentional (new requirement): update the .tf files to match, then apply

■ *Drift is a symptom of teams making ad-hoc console changes. Enforce IaC-only changes via SCPs or IAM policies.*

Q9. How do you update a production resource with zero downtime?

Strategy depends on the resource type:

- Use create_before_destroy lifecycle rule for stateless resources (EC2, ECS tasks)
- For databases: enable multi-AZ, use blue-green deployment or read replica promotion
- For ALB/Route53: use weighted routing to shift traffic gradually
- For EKS: rolling updates via Helm with PodDisruptionBudgets

```
resource "aws_instance" "web" {
  ...
  lifecycle {
    create_before_destroy = true
  }
}
```

■ *Some resource changes force replacement regardless (e.g., RDS engine version). Always check plan output for 'forces replacement'.*

Q10. How do you safely destroy only one specific resource?

```
# Target a specific resource:
terraform destroy -target=aws_instance.web

# Or for a module resource:
terraform destroy -target=module.network.aws_vpc.main

# Always preview first:
terraform plan -destroy -target=aws_instance.web
```

Use -target sparingly. It intentionally bypasses Terraform's dependency graph, which can leave dependent resources in broken states.

■ *After a targeted destroy, always run a full terraform plan to check for unintended state inconsistencies.*

SECTION 3 — Multi-Environment Design

Q11. How would you design the same infrastructure for Dev, QA, and Production?

Three common patterns — each with real trade-offs:

- Pattern A — Separate directories per env (most explicit, most duplication):

```
infra/
  dev/     # dev main.tf, variables.tf
  qa/
  prod/
```

- Pattern B — Workspaces (single codebase, multiple state files; risky for prod isolation):

```
terraform workspace new dev
terraform workspace select prod
```

- Pattern C — Terragrunt (DRY wrapper, separate state per env, recommended at scale):

```
root.hcl         # shared config
dev/main.hcl
prod/main.hcl    # each calls same modules
```

For most teams: separate directories + shared modules is the safest balance between isolation and DRY code.

■ *Never share a state file between environments. One bad prod apply can corrupt your dev understanding of state.*

Q12. When would you use Workspaces vs separate state files?

Workspaces — appropriate when:

- Infrastructure is identical across environments (same resources, only values differ)
- Small team, low risk of someone running apply on wrong workspace accidentally
- Quick ephemeral environments (feature branches, review apps)

Separate state files — preferred when:

- Prod needs strict isolation — wrong workspace selection can't destroy prod
- Environments differ structurally (prod has WAF, dev doesn't)
- Different teams/roles manage different environments

■ *Workspaces are often misused for prod/non-prod isolation. The blast radius of terraform destroy in the wrong workspace is severe.*

Q13. How do you manage different variable values for different environments?

Several complementary techniques:

- Use .tfvars files per environment: dev.tfvars, qa.tfvars, prod.tfvars
- Pass them at runtime: terraform apply -var-file=prod.tfvars
- Use environment variables: TF_VAR_instance_type=t3.large
- Store secrets in AWS SSM Parameter Store or Secrets Manager and reference via data sources

```
# dev.tfvars
instance_type = "t3.micro"
replicas      = 1

# prod.tfvars
instance_type = "m5.xlarge"
replicas      = 3
```

■ *Never commit prod.tfvars with secrets to Git. Use .gitignore and inject secrets at CI/CD runtime.*

Q14. How do you keep environments isolated?

Isolation operates at multiple levels:

- Account-level: Separate AWS accounts per env (recommended) — eliminates blast radius

- State-level: Separate S3 buckets/keys per environment — no shared state
- Network-level: No VPC peering between prod and dev
- IAM-level: Separate roles per environment with least-privilege policies
- CI/CD-level: Separate pipelines — prod deploys require manual approval

■ *The strongest isolation is separate AWS accounts. SCPs at the org level can enforce what resources are allowed in each account.*

SECTION 4 — Modules & Reusability

Q15. How do you design reusable Terraform modules?

A well-designed module follows these principles:

- Single responsibility — one module does one thing (e.g., 'vpc', 'eks-cluster', 'rds')
- All inputs via variables.tf, all outputs via outputs.tf — no hardcoded values
- Sensible defaults for optional variables so callers don't need to set everything
- Semantic versioning — modules are tagged so consumers can pin to stable versions

```
# Module structure:
modules/eks-cluster/
  main.tf         # EKS resource definitions
  variables.tf    # Inputs: cluster_name, node_type, min_size...
  outputs.tf      # Outputs: cluster_endpoint, oidc_issuer...
  versions.tf     # Required Terraform/provider versions
  README.md       # Document all inputs/outputs
```

Q16. How do you version modules across multiple teams?

Use Git tags + versioned module sources:

```
module "eks" {
  source  = "git::https://github.com/org/tf-modules//eks?ref=v2.1.0"
  # Or from Terraform Registry:
  # source  = "org/eks/aws"
  # version = "~> 2.1"
}
```

- Tag every release: git tag v2.1.0 && git push origin v2.1.0
- Maintain a CHANGELOG.md — breaking changes need a major version bump
- Pin module versions in consumer code — avoid source = ...?ref=main (unstable)

■ *~> 2.1 means '>= 2.1.0, < 3.0.0' — useful for patch updates without breaking changes.*

Q17. How do you share modules securely?

Options by access model:

- Private Git repo with SSH keys or GitHub App token in CI/CD
- Terraform Cloud/Enterprise Private Registry — built-in versioning and access control
- S3 bucket (zip archive) — simple but no versioning UI
- Artifactory or Nexus — for enterprises with existing artifact management

Secure the module repo like any production codebase: branch protection, required reviews, no direct main pushes.

■ *Avoid sourcing modules from public GitHub repos in production — you don't control upstream changes or deletions.*

Q18. How do you structure modules for a microservices architecture?

Two-layer module hierarchy works well:

- Base modules (low-level): vpc, iam-role, rds, s3-bucket, ecr-repo
- Service modules (composite): compose base modules into a 'microservice stack'

```
# Service module for each microservice:
module "payment-service" {
  source       = "./modules/ecs-service"
  service_name = "payment"
  image_uri    = var.payment_image
  cpu          = 512
  memory       = 1024
  port         = 8080
}
```

- One state file per service or one per environment — avoid a monolithic state with all 20 services

■ *Large monolithic states slow down plan/apply for unrelated changes and increase blast radius on state corruption.*

SECTION 5 — Import & Migration

Q19. An AWS infrastructure already exists manually. How do you bring it under Terraform?

Systematic import process:

- Step 1: Audit existing resources — list all resources to be managed
- Step 2: Write matching .tf resource definitions (configurations must match reality)
- Step 3: Run terraform import for each resource to pull it into state
- Step 4: Run terraform plan — it should show 'No changes' if config matches actual
- Step 5: Fix any attribute mismatches, re-plan until clean

```
# Import an existing EC2 instance:
terraform import aws_instance.web i-0abc123def456

# Import an existing S3 bucket:
terraform import aws_s3_bucket.logs my-existing-bucket-name

# Terraform 1.5+ import block (preferred):
import {
  to = aws_instance.web
  id = "i-0abc123def456"
}
```

■ *Tools like Terraformer or cf2tf can auto-generate .tf files from existing infra, but always review output — it's a starting point, not production-ready code.*

Q20. What are the limitations of terraform import?

Key limitations to know in interviews:

- Does not generate .tf configuration — you must write the resource block manually first
- One resource at a time (CLI) — no bulk import of entire infrastructure
- Not all resources are importable — some Terraform resources lack import support
- Complex nested resources (e.g., inline security group rules) may import incorrectly
- Imported state may differ from real config — plan will show changes if your .tf doesn't exactly match
- No way to import data sources — only managed resources

■ *Terraform 1.5 introduced the import block, which is declarative and version-controlled. It's better than the CLI import command for team settings.*

Q21. How do you migrate resources without recreating them?

The core principle: update state to reflect the rename/move without destroying real infrastructure.

- Rename within same state: use moved block or terraform state mv
- Move between modules: terraform state mv module.old.aws_instance.web module.new.aws_instance.web

- Move between state files: terraform state mv with -state-out flag

```
# Rename resource without recreation (Terraform 1.1+):
moved {
  from = aws_instance.old_name
  to   = aws_instance.new_name
}

# CLI approach:
terraform state mv aws_instance.old_name aws_instance.new_name
```

After the move, run terraform plan — it must show zero changes before you proceed. Any planned changes mean the config doesn't match state.

■ *The moved block is declarative and reviewable in PRs. Use it over CLI state mv whenever possible.*

Quick Reference Summary

Topic	Key Commands / Concepts
State Locking	S3 + DynamoDB backend, terraform init
Corruption Recovery	S3 versioning, .tfstate.backup, terraform import
Drift Detection	terraform plan, terraform refresh, Driftctl
Partial Apply	terraform plan → fix root cause → terraform apply
Environment Design	Separate state files, separate AWS accounts
Module Versioning	Git tags, ~> version constraint, private registry
Import	terraform import / import block (TF 1.5+)
Zero-Downtime	create_before_destroy, blue-green, rolling updates
Resource Migration	moved block (TF 1.1+), terraform state mv